Investment Trading Signal using Regression

Slide Type Slide

Imported Modules

Slide Type Fragment

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import statsmodels.formula.api as smf
import warnings
warnings.filterwarnings("ignore")
```

Slide Type Slide

Benchmark with Linear Regression

Slide Type Slide

If we have more than 1 predictors in linear regression model, it is called multiple linear regerssion. Simple linear regression can easily be extended to include multiple features. This is called **multiple linear regression**:

$$y = \beta_0 + \beta_1 x_1 + \ldots + \beta_k x_k + \epsilon$$

Slide Type Slide

Assumptions about noise still holde:

- for different values of features, noises are independent and have equal variance σ
- noises are normal
- means of noises are zeroes.

Besides:

- all features are independent.

These assumptions are important, without which, statistical properties of coefficient and prediction will not hold. That is, we still can get least square equation. $\hat{y} = b_0 + b_1 x_1 + \ldots + b_k x_k$, but we cannot evaluate variance of the model.

In many datasets, these assumptions are not true originally. But we can transform data so that assumptions are approximately hold.

Slide Type Slide

Loading and Munging data

Slide Type Slide

```
data = pd.read_csv('data/indicepanel.csv',index_col=0)
data.head()
```

	nikkei	aord	nyse	dax	ftse	hangseng	sp500	djia
2013-09-06	13860.81	5144.0	9420.3480	8234.98	6547.33	22621.22	1655.08	14937.48
2013-09-09	14205.23	5179.4	9420.3480	8234.98	6530.74	22750.65	1655.08	14937.48
2013-09-10	14423.36	5198.9	9539.9320	8276.32	6583.99	22976.65	1671.71	15063.12
2013-09-11	14425.07	5230.6	9620.7100	8446.54	6588.43	22937.14	1683.99	15191.06
2013-09-12	14387.27	5238.2	9655.3774	8495.73	6588.98	22953.72	1689.13	15326.60

Slide Type: Slide

First we transform all price into log Return
r = (P(n+1) - P(n))/p(n)
1+r = exp(R)
P(n+1) / P(n) = exp(R)
ln(P(n+1)) - ln(P(n)) = R
r ~ ln(1+r) = R

Slide Type: Slide

```
ReturnPanel=pd.DataFrame()
ReturnPanel['sp500']=np.log(data['sp500'])-np.log(data['sp500'].shift(1))
ReturnPanel['nikkei']=np.log(data['nikkei'])-np.log(data['nikkei'].shift(1))
ReturnPanel['aord']=np.log(data['aord'])-np.log(data['aord'].shift(1))
ReturnPanel['nyse']=np.log(data['nyse'])-np.log(data['nyse'].shift(1))
ReturnPanel['dax']=np.log(data['dax'])-np.log(data['dax'].shift(1))
ReturnPanel['ftse']=np.log(data['ftse'])-np.log(data['ftse'].shift(1))
ReturnPanel['hangseng']=np.log(data['hangseng'])-np.log(data['hangseng'].shift(1))
ReturnPanel['djia']=np.log(data['djia'])-np.log(data['djia'].shift(1))
ReturnPanel=ReturnPanel.dropna(axis=0)
ReturnPanel.head()
```

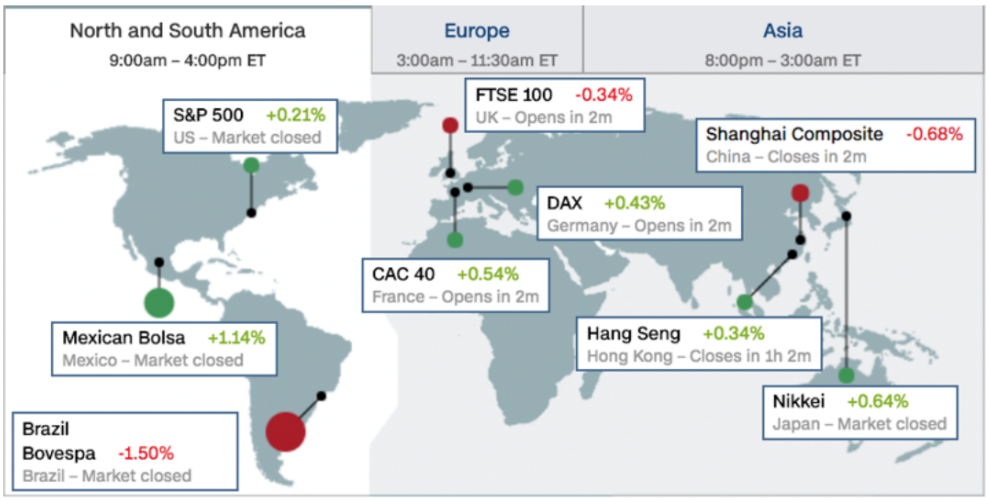
Slide Type: Slide

Check the following graph, we know that, We can not use log return of DAX, FTSE, NYSE, DJIA to predict the log return of SP500 on the same day. But we can their returns on previous days.

Slide Type: Slide

Index	Country	Closing Time (EST)	Hours Before S&P Close
All Ords	Australia	0100	15
Nikkei 225	Japan	0200	14
Hang Seng	Hong Kong	0400	12
DAX	Germany	1130	4.5
FTSE 100	UK	1130	4.5
NYSE Composite	US	1600	0
Dow Jones Industrial Average	US	1600	0
S&P 500	US	1600	0

Slide Type: Slide



Slide Type Slide

```
dataMatrix=pd.DataFrame()
dataMatrix['sp500']=ReturnPanel['sp500']
dataMatrix['sp500_lag']=ReturnPanel['sp500'].shift(1)
dataMatrix['nikkei']=ReturnPanel['nikkei']
dataMatrix['nikkei_lag']=ReturnPanel['nikkei'].shift(1)
dataMatrix['aord']=ReturnPanel['aord']
dataMatrix['aord_lag']=ReturnPanel['aord'].shift(1)
dataMatrix['hangseng']=ReturnPanel['hangseng']
dataMatrix['hangseng_lag']=ReturnPanel['hangseng'].shift(1)
dataMatrix['nyse_lag']=ReturnPanel['nyse'].shift(1)
dataMatrix['dax_lag']=ReturnPanel['dax'].shift(1)
dataMatrix['ftse_lag']=ReturnPanel['ftse'].shift(1)
dataMatrix['djia_lag']=ReturnPanel['djia'].shift(1)
dataMatrix=dataMatrix.dropna(axis=0)
dataMatrix.index=pd.to_datetime(dataMatrix.index)
dataMatrix.head(2)
```

	sp500	sp500_lag	nikkei	nikkei_lag	aord	aord_lag	hangseng	hangseng_lag	nyse_lag	dax_lag	ftse_lag	djia_lag
2013-09-10	0.009998	0.000000	0.015239	0.024545	0.003758	0.006858	0.009885	0.005705	0.000000	0.000000	-0.002537	0.000000
2013-09-11	0.007319	0.009998	0.000119	0.015239	0.006079	0.003758	-0.001721	0.009885	0.012614	0.005007	0.008121	0.008376

Slide Type Slide

The first column is reponse and the rest columns are predictors. Totally we have 11 predictors and 974 days' data. We need to split the data into train and test.

Slide Type Slide

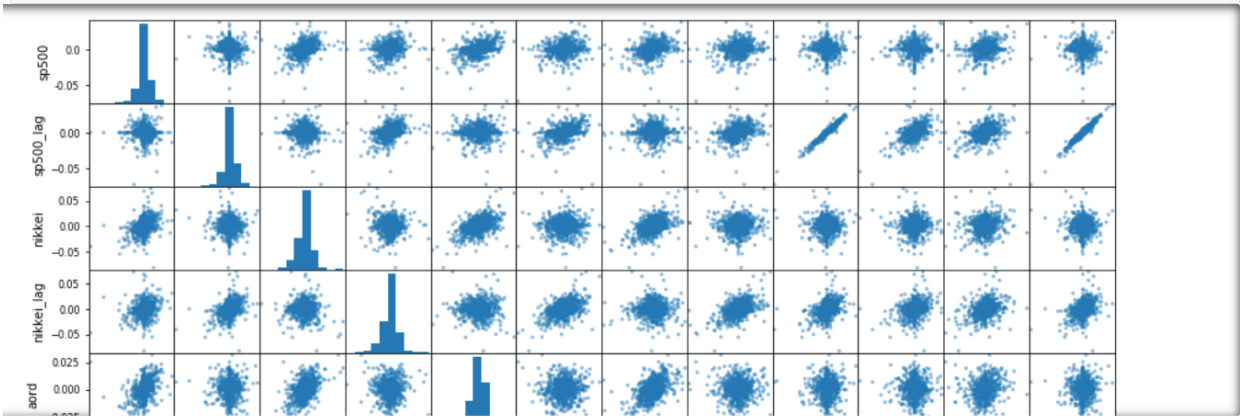
```
Train=dataMatrix.iloc[:780,:]
Test=dataMatrix.iloc[780:,:]
```

Slide Type Slide

Descriptive Statistics

Slide Type Slide

```
from pandas.plotting import scatter_matrix
sm=scatter_matrix(Train,figsize=(15,15))
```



Slide Type Slide

```
dataMatrix.corr() # correlation only evaluate the linear relationship, not non-linear
```

	sp500	sp500_lag	nikkei	nikkei_lag	aord	aord_lag	hangseng	hangseng_lag	nyse_lag	dax_lag	ftse_lag	djia_lag
sp500	1.000000	-0.025691	0.373530	0.216522	0.368018	0.196603	0.287842	0.202194	-0.015715	0.043801	0.302494	-0.016596
sp500_lag	-0.025691	1.000000	-0.033395	0.372875	0.024879	0.367868	0.033688	0.287819	0.973362	0.584991	0.343754	0.968794
nikkei	0.373530	-0.033395	1.000000	-0.052714	0.498112	0.049907	0.483824	0.011413	-0.028551	-0.017755	0.140160	-0.025052
nikkei_lag	0.216522	0.372875	-0.052714	1.000000	0.011187	0.498459	0.023068	0.483609	0.366960	0.298491	0.324760	0.368278
aord	0.368018	0.024879	0.498112	0.011187	1.000000	0.016371	0.511812	0.013583	0.034519	-0.005601	0.159642	0.027488
aord_lag	0.196603	0.367868	0.049907	0.498459	0.016371	1.000000	0.029091	0.511833	0.375884	0.290009	0.364569	0.356226
hangseng	0.287842	0.033688	0.483824	0.023068	0.511812	0.029091	1.000000	-0.021749	0.045008	0.046306	0.137066	0.028920
hangseng_lag	0.202194	0.287819	0.011413	0.483609	0.013583	0.511833	-0.021749	1.000000	0.309392	0.187690	0.369751	0.285290
nyse_lag	-0.015715	0.973362	-0.028551	0.366960	0.034519	0.375884	0.045008	0.309392	1.000000	0.622415	0.367579	0.953741
dax_lag	0.043801	0.584991	-0.017755	0.298491	-0.005601	0.290009	0.046306	0.187690	0.622415	1.000000	0.354429	0.586546
ftse_lag	0.302494	0.343754	0.140160	0.324760	0.159642	0.364569	0.137066	0.369751	0.367579	0.354429	1.000000	0.341045
djia_lag	-0.016596	0.968794	-0.025052	0.368278	0.027488	0.356226	0.028920	0.285290	0.953741	0.586546	0.341045	1.000000

Slide Type Slide

From correlation matrix,we can find that response 'sp500' highly correlates with nikkei,nikkei_lag, aord, aord_lag, hangseng , hangseng_lag, ftse_lag. Besides, the correlation between sp500 and these indices is higher than those btween sp500 and their lag.

High correlations also exist among these strong predictors. Next we first use these strong predictors to build a multiple linear regression.

Slide Type Slide

Build linear regression model

Slide Type Slide

```
# create a fitted model with all three features
lm= smf.ols(formula='sp500 ~ nikkei+nikkei_lag+aord+aord_lag+hangseng+hangseng_lag+ftse_lag ', data=Train).fit()
```

Slide Type Slide

```
lm.summary(alpha=0.05)
```

OLS Regression Results

Dep. Variable:	sp500	**R-squared:**	0.269
Model:	OLS	**Adj. R-squared:**	0.263
Method:	Least Squares	**F-statistic:**	40.68
Date:	Thu, 24 Oct 2019	**Prob (F-statistic):**	8.01e-49
Time:	09:56:32	**Log-Likelihood:**	2696.0
No. Observations:	780	**AIC:**	-5376.
Df Residuals:	772	**BIC:**	-5339.
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	0.0003	0.000	0.970	0.332	-0.000	0.001
nikkei	0.1437	0.023	6.140	0.000	0.098	0.190
nikkei_lag	0.0796	0.023	3.391	0.001	0.034	0.126
aord	0.2124	0.040	5.336	0.000	0.134	0.291
aord_lag	0.0215	0.040	0.532	0.595	-0.058	0.101
hangseng	0.0257	0.029	0.876	0.381	-0.032	0.083
hangseng_lag	0.0513	0.030	1.718	0.086	-0.007	0.110
ftse_lag	0.1454	0.031	4.648	0.000	0.084	0.207

Omnibus:	195.352	**Durbin-Watson:**	2.502
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	2194.850
Skew:	-0.787	**Prob(JB):**	0.00
Kurtosis:	11.066	**Cond. No.**	160.

Slide Type Slide

Practical Standards

Slide Type Slide

We use model to build trading signal. Performance of trading based on your signal is the practical standard of your models.

Slide Type Slide

- Drawdown:the percentage decline in the strategy from the historical peak profit at each point in time.

Slide Type Slide

- Average Daily return

  $$(1+r)^{days} = 1 + R$$

  where r is average Daily return and R is total return.

Slide Type Slide

- Sharpe ratio: The ratio measures the excess return (or risk premium) per unit of deviation in an investment asset or a trading strategy, typically referred to as risk, named after William F. Sharpe

  $$SR = \frac{E(R_a - R_b)}{\sqrt{Var(R_a - R_b)}}$$

  Suppose the daily Shape ratio is SR_{Day}. Then yearly Shape ratio is

  $$SR_{Year} = \sqrt{250} SR_{Day}$$

Every year has 250 days trading. (assume daily performance is independent)

Var(R1 + R2) = Var(R1)+Var(R2)+2Cov(R1,R2)
Overestimate annual Sharpe ratio if positively correlated.

Slide Type: Slide

Trading strategy

Slide Type: Fragment

- Benchmark Strategy I: Buy and Hold

Slide Type: Fragment

- Benchmark Strategy II: Persistent strategy, if last day's return is postive , we buy today, otherwise we short sell

Slide Type: Fragment

- Signal-based Strategy: If Signal is postive , we buy, otherwise we short sell.

Slide Type: Slide

What we trade

Slide Type: Fragment

S&P 500 ETF (Exchanged traded fund)- SPY

Slide Type: Slide

Signal-based Strategy

Slide Type: Slide

```
Trade_Train=pd.DataFrame()
Trade_Train['Price']=Train['Price']
Trade_Train['Signal']=Train_predict
Trade_Train.head()
```

	Price	Signal
2014-07-09	1963.71	-0.004444
2014-07-10	1972.83	-0.002017
2014-07-11	1964.68	-0.001045
2014-07-14	1964.68	0.002429
2014-07-15	1977.10	0.003539

Slide Type: Slide

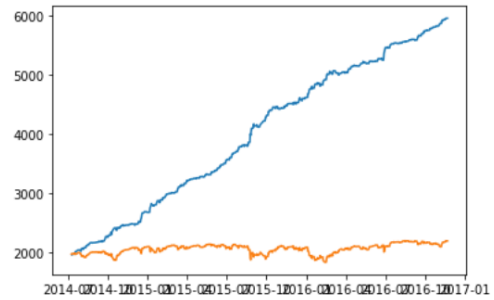
```
Trade_Test=pd.DataFrame()
Trade_Test['Price']=Test['Price']
Trade_Test['Signal']=Test_predict
Trade_Test.head()
```

	Price	Signal
2016-11-22	2198.18	0.003352
2016-11-24	2204.72	0.004544
2016-11-25	2204.72	0.003104
2016-11-28	2204.72	-0.000277
2016-11-29	2201.72	-0.000763

Slide Type: Slide

```
Trade_Train['Order']=[1 if sig>0 else -1 for sig in Trade_Train['Signal']  ] # 1: buy, -1: short sell
Trade_Train['Price_change']=Trade_Train['Price']-Trade_Train['Price'].shift(1) # today's price - yesterday's price
Trade_Train['Profit']=Trade_Train['Price_change']*Trade_Train['Order']
print(Trade_Train['Profit'].sum())
Trade_Train['Wealth']=np.cumsum(Trade_Train['Profit'])+Trade_Train['Price'][0]
plt.plot(Trade_Train['Wealth'])
plt.plot(Trade_Train['Price']) # performance if you hold
```

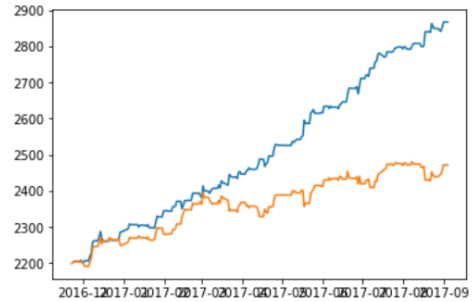
```
3989.610000000002

[<matplotlib.lines.Line2D at 0x1a26bcc898>]
```



Slide Type: Fragment

```
Trade_Test['Order']=[1 if sig>0 else -1 for sig in Trade_Test['Signal']  ]
Trade_Test['Price_change']=Trade_Test['Price']-Trade_Test['Price'].shift(1)
Trade_Test['Profit']=Trade_Test['Price_change']*Trade_Test['Order']
print(Trade_Test['Profit'].sum())
Trade_Test['Wealth']=np.cumsum(Trade_Test['Profit'])+Trade_Test['Price'][0]
plt.plot(Trade_Test['Wealth'])
plt.plot(Trade_Test['Price'])
print(Trade_Test['Profit'].sum())
```

```
669.3899999999985
669.3899999999985
```



Slide Type Slide

sharpe ratio

Slide Type Fragment

```
Trade_Train.head()
# daily return
Trade_Train['Return']=(Trade_Train['Wealth']
                       -Trade_Train['Wealth'].shift(1))/Trade_Train['Wealth'].shift(1)
dailyr=Trade_Train['Return'].dropna()
print("daily sharpe ratio is ", dailyr.mean()/dailyr.std(ddof=1))
print("yearly sharpe ratio is", (250**0.5)*dailyr.mean()/dailyr.std(ddof=1))
```

```
daily sharpe ratio is  0.3905239545988772
yearly sharpe ratio is 6.1747258869431985
```

Slide Type Fragment

```
Trade_Test.head()
# daily return
Trade_Test['Return']=(Trade_Test['Wealth']
                       -Trade_Test['Wealth'].shift(1))/Trade_Test['Wealth'].shift(1)
dailyr=Trade_Test['Return'].dropna()
print("daily sharpe ratio is ", dailyr.mean()/dailyr.std(ddof=1))
print("yearly sharpe ratio is", (250**0.5)*dailyr.mean()/dailyr.std(ddof=1))
```

```
daily sharpe ratio is  0.35527189022345096
yearly sharpe ratio is 5.617341808697059
```

Slide Type Slide

Drawdown

Slide Type Slide

```
# list comprehension
Trade_Train['Peak']=[Trade_Train['Wealth'].loc[:any_index].max()
                     for any_index in Trade_Train.index ]
Trade_Train['Drawdown']=(Trade_Train['Wealth']-Trade_Train['Peak'])/Trade_Train['Peak']
print("Maximum drawdown rate is", -Trade_Train['Drawdown'].min())
```

```
Maximum drawdown rate is 0.017250046739649576
```

Slide Type Fragment

```
# list comprehension
Trade_Test['Peak']=[Trade_Test['Wealth'].loc[:any_index].max()
                    for any_index in Trade_Test.index ]
Trade_Test['Drawdown']=(Trade_Test['Wealth']-Trade_Test['Peak'])/Trade_Test['Peak']
print("Maximum drawdown rate is", -Trade_Test['Drawdown'].min())
```

```
Maximum drawdown rate is 0.011887378131421152
```